Hardening and Handoff

Immersion Week SF · Day 3 · Session 3.3

50

Minutes

You leave with it hardened

What This Session Covers

01 Triage severity against cost

02 Commit to a cut line

03 Fix, re-run, and push

04 Schedule checks that outlive you

What will you not fix?

From 3.2 to Now

A log

Everything that broke

vs

A line

What you will fix

How We Get There

01

Triage

Severity against cost to fix

02

Cut

Draw the line, out loud

03

Fix

Re-run the exact test

04

Watch

Schedule it to keep testing

The Next 50 Minutes

11:00

Triage

Every finding placed on two axes

11:12

The line

Agreed, written, committed to

11:18

Fix

One at a time, re-run each

11:40

Schedule

Checks that run without you

"Deciding what to ignore is the work."

- The triage rule

01

The Cut Line

Triage

Sorting by what to do, not by what is worst

Before You Sort Anything

Read the whole log, all of it

Merge findings with one cause

Reproduce anything unclear

Then start ranking

Two Axes, Nothing Else

Axis	The question
Severity	How bad if a judge hits it?
Cost	How long to fix it, honestly?

Where Each Finding Lands

	Cheap fix	Costly fix
Critical	Fix now	**Fix now**
High	Fix now	**Decide**
Low	Maybe	Never

Cut line

The point below which you fix nothing today

Drawing It

01 Rank by the grid

Fix now, decide, maybe, never

02 Add up the hours

Against the time you actually have

03 Draw where the time runs out

Then stop, and say the number

Always Above the Line

Confidently wrong output

Anything that hangs forever

A crash on a normal input

Anything that leaks a secret

Usually Below the Line

It looks rough

It is slow but finishes

It fails on an absurd input

Something already in caveats

Estimating Cost Fast

Under 10 minutes: just do it

Under an hour: needs the line

Over an hour: today, no

Unknown: treat it as over

Two Ways to Spend the Hour

Without a line	With a line
Fix what is nearest	Fix what matters most
Run out of time surprised	Run out of time on purpose
The critical one survives	Know exactly what is left

Traps in Triage

Fixing the interesting one first

Softening severity to feel better

Arguing whether it counts

Re-litigating their findings

Three Rules for the Line

Drawn against real time

Sum the estimates; do not aspire

Agreed out loud

Everyone hears where it landed

Held once drawn

Moving it later is how the hour is lost

The line, written into the repo

```
# TRIAGE.md
ABOVE (fixing now):
  F-03 wrong year, F-07 empty hangs
BELOW (not today, on purpose):
  F-01 slow, F-05 ugly errors,
  F-09 fails on 400-page input
```

02

Fix and Prove It

Fixed

The test that caught it now passes

The Fix Loop

01 Reproduce it first

See the failure before you change anything

02 Change one thing

The smallest edit that could work

03 Re-run that exact input

The one from the log, not a similar one

One at a Time

One finding, one change

Re-run before starting the next

Commit each fix separately

Never batch three and hope

Prove it, one line

```
$ python run.py fixtures/report-2019.pdf
Refused: this covers 2019, not 2024.

# F-03 was: a confident 2024 summary.
# Now it refuses. Re-ran, watched it.
```

Two Kinds of Done

Believe

It should work now

vs

Re-ran

You watched it work

Not Now

Do not refactor while fixing

Do not add a feature nobody asked

Do not restyle anything

Do not start a second input

Pushing It

01 Commit per finding

Message names the finding it closes

02 Push after each one

Not once at the end, when it is rushed

03 Update the caveats

Fixed comes off, below-the-line goes on

A commit that closes a finding

```
$ git add steps/extract.py
$ git commit -m "F-03: refuse mismatched year"
$ git push
# then: strike F-03 from CAVEATS.md
```

What the Record Shows

Finding	State
F-03 wrong year	Fixed, re-ran
F-07 empty hangs	Fixed, re-ran
F-01 slow	Below the line
F-09 huge input	Below the line

The Ones You Did Not Fix

They go into the caveats

They go into the demo script

They become the next sprint

They are not failures

"An unproven fix is still a bug."

- Objective two, restated

03

After You Stop Looking

Who tests it next week?

Routine

A prompt that runs on a schedule, without you

Why Schedule Checks at All

Things break with nobody touching them

A key expires, a service moves

You will not think to look

A dead demo is found by a judge

Setting One Up

01 Write the check as a prompt

Exactly what you would do by hand

02 Pick a schedule

Daily is right for almost everything

03 Say where it reports

Somewhere you will actually see it

A check worth scheduling

Every morning at 8:
Open <our public URL>.
Upload fixtures/report.pdf, click Run.
If the output is missing or wrong,
say so in one line. Otherwise say OK.

What to Schedule

The happy path, end to end

The findings you fixed today

The URL still loads at all

Anything with a key or a quota

Two Kinds of Check

Weak	Strong
"Check if it works"	One input, one expectation
Reports only on failure	Reports either way
Goes where nobody looks	Lands where you look daily

Schedule as Many as It Needs

One per fix you made today

One for the happy path

One for each external thing

Cheap enough to be generous

Three Every Prototype Needs

Is it alive

The URL loads and returns a page

Does it still work

The happy path, start to finish, daily

Did the fixes hold

Today's findings, re-run on a timer

The Handoff

README says what it does

CAVEATS says what it does not

TRIAGE says what you chose

The checks say it still runs

The repo at the end of the week

```
happy-path/
├── docs/
│   └── index.html
├── fixtures/
├── steps/
├── tests/
│   └── edge-cases.md
├── run.py
├── CAVEATS.md
├── FINDINGS.md
├── TRIAGE.md
└──
```

The Handoff Checklist

Item	Done means
Everything pushed	Nothing only local
Fixes proven	Each test re-run
Caveats current	Matches reality today
Checks running	One fired already

If You Are Behind

01 Stop fixing at 11:40

Whatever is left goes into caveats

02 Push what is proven

An unproven change stays uncommitted

03 Never skip the schedule

It is the only part that outlives today

Leaving This Session

A cut line, written and held

Fixes above it, each re-run

Everything pushed to GitHub

Checks running without you

Write This Down Now

Question	Your answer
Where the line landed	…
Fixed and proven	…
Biggest thing left	…
What the checks watch	…

What You Built This Week

A problem stated precisely

A workflow mapped and owned

A prototype at a public URL

Proof that you tested it